```
implicit val bFormat: RootJsonFormat[Cargo] =
              jsonFormat2(Cargo("bigBag",1))      ❶
```

❶ [Cargo] is composed of two serializable basic classes: String and Int.

`jsonFormat2` is an `akka-http-spray-json` method that signals with its name that the `B` constructor has two parameters. However, they don't have to be basic parameters as long as they have their own formatters. You can see that `B` has two parameters, a `String` and an `Int`. There are `jsonFormat3`, `jsonFormat4`, and so forth.

Once `spray` knows how many parameters the constructor has, it can create the necessary formatter. There are two types: `JsonFormat` and `RootJsonFormat`. Since formatters can be composed just like the classes they format, a class `B(f1: C, f2: D)` needs one formatter for `B`, another for `C`, and another for `D`. Here, `B` is the root formatter and must be a `RootJsonFormat`. The other two can be `JsonFormatter`s.

In practice, this distinction is not very important, and it may be better if you use `RootJsonFormat` everywhere and forget which object is the root. This way, you avoid compilation errors when you need a `JsonFormat` formatter that acts as root. This is what `spray-json` does by default to give you flexibility: it sets every formatter to `RootJsonFormat`. A counterargument is that you want to align the formatting with the hierarchy in your domain to ensure that the business modeling is followed.

Now that you know about routes, directives, and marshallers, you can learn how to use them to connect to an actor from outside the